Universidad Rafael Landívar

Ciencia de Datos · Segundo semestre 2026

Ing. Max Cerna

Proyecto 2 (Fase 2)

Red Metropolitana

¿Quién deja de usar la red?

Integrantes:
Yasmely Dubraska Fariñas Aldana 1329223

Jose Rodrigo Herrera Muralles 1042619

Walter José Herrera Salvatierra 1109722

9 de noviembre de 2026

Resumen

En la Fase 1 dejamos lista la etiqueta, la población, las 23 features por fecha de corte (sin fuga) y la regresión logística con sus líneas base. En esta fase comparamos ensembles con los mismos 5 folds, elegimos el umbral con el escenario de costos de la Agencia, evaluamos el modelo final una sola vez en el corte B, segmentamos a los usuarios y dejamos el modelo en producción con predecir.py, su ficha y las predicciones del corte C.

Qué medimos	Resultado
Mejor modelo (validación, 5 folds de A)	HistGradientBoosting: precisión promedio 0.4388 ± 0.0113 (línea base días desde el último viaje: 0.3892; logística L2: 0.4129)
Umbral	0.33 (equilibrio teórico 1/3); en A: G = 10 · 2,815 − 5 · 3,199 = 28,150 − 15,995 = Q12,155
Test B (una sola vez)	2026-11-08 12:21, commit 556830f: precisión promedio 0.3905 (línea base 0.3482); G = 10 · 2,653 − 5 · 3,652 = 26,530 − 18,260 = Q8,270
Segmentos	k = 4: ocasionales que se enfrían (21.3 % de abandono), Aerómetro (14.4 %), regulares (3.9 %), intensivos (1.8 %)
Corte C	57,701 personas, 7,268 a contactar (predicciones/predicciones_corte_2026-08-30.csv)
Invariancia (ensayo con B)	Con 30 días más de datos el CSV sale idéntico (mismo SHA-256)

Dónde está cada script

Punto	Script
2.1	proyecto2/ensemble.py (Random Forest y HistGradientBoosting con los folds de la Fase 1, sesgo-varianza e importancias); proyecto2/datos_modelo.py (piezas compartidas)
2.2	proyecto2/umbral.py (curva de ganancia con predicciones fuera de fold de A); proyecto2/entrenar_final.py (modelo con todo A); proyecto2/test_b.py (evaluación única en B)
2.3	proyecto2/segmentos.py (codo, silueta, KMeans y PCA)
2.4	predecir.py; modelos/modelo_abandono_v1.0.0.joblib; docs/proyecto2/ficha-modelo.md; proyecto2/invariancia.py (ensayo)
Todos	flows/pipeline.py --desde-cero (agrega 2.1, 2.2, 2.3 y la predicción de C); proyecto2/bitacora.md (fecha y commit de cada corrida)

2.1 Ensemble y sesgo-varianza

Entrenamos dos ensembles, un bagging (Random Forest) y un boosting (HistGradientBoostingClassifier), con los mismos 5 folds estratificados del corte A que usamos en la Fase 1: el script los lee de folds-corte-a.csv y comprueba que son los de StratifiedKFold con la semilla 2026. El preprocesamiento va dentro del Pipeline, así que se ajusta solo con el entrenamiento de cada fold. En el boosting, learning_rate cumple el papel de α; probamos dos valores y dos tamaños de árbol, con parada temprana sobre un 10 % del entrenamiento del fold. Ningún paso de esta sección lee el corte B.

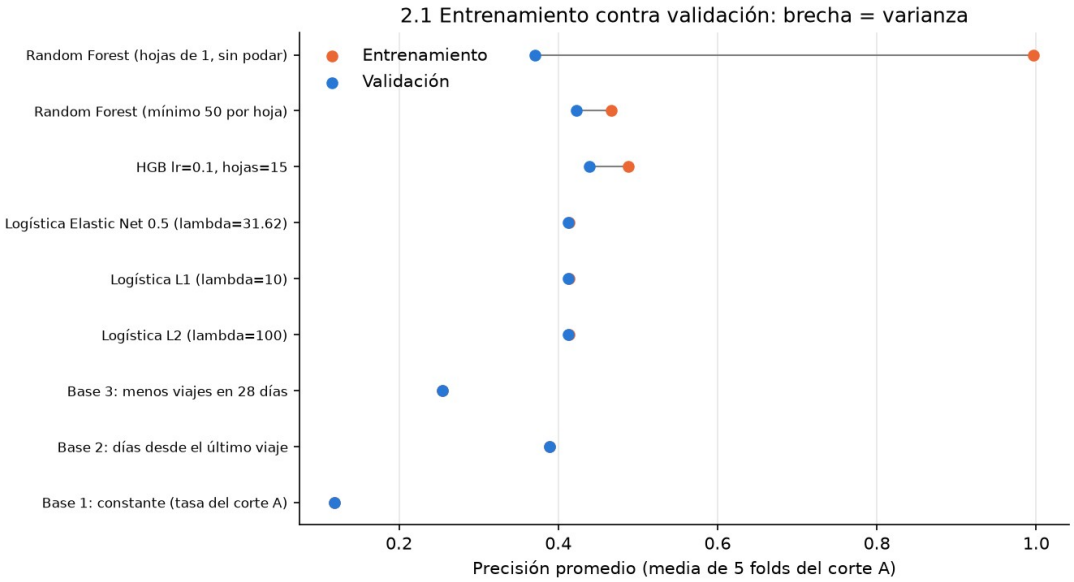
Tabla de modelos

Modelo	Precisión prom. entrenamiento	Precisión prom. validación (media)	Desv. estándar	ROC-AUC validación
Base 1: constante (tasa del corte A)	0.1193	0.1193	0.0000	0.5000
Base 2: días desde el último viaje	0.3889	0.3892	0.0078	0.8229
Base 3: menos viajes en 28 días	0.2540	0.2541	0.0063	0.7426
Logística L2 (lambda=100)	0.4137	0.4129	0.0112	0.8312
Logística L1 (lambda=10)	0.4133	0.4124	0.0114	0.8315
Logística Elastic Net 0.5 (lambda=31.62)	0.4134	0.4126	0.0113	0.8316
HGB lr=0.1, hojas=15	0.4884	0.4388	0.0113	0.8446
Random Forest (mínimo 50 por hoja)	0.4669	0.4226	0.0113	0.8421
Random Forest (hojas de 1, sin podar)	0.9970	0.3705	0.0137	0.8133

Todas las filas usan los mismos 5 folds. Las líneas base no aprenden nada, así que su métrica de entrenamiento es la misma que la de validación. Configuraciones del boosting probadas (precisión promedio de validación): lr 0.05 con 15 hojas 0.4380; lr 0.05 con 31 hojas 0.4367; lr 0.1 con 15 hojas 0.4388; lr 0.1 con 31 hojas 0.4345 (entregables/proyecto2/2.1-configuraciones.csv).

Sesgo-varianza

Modelo	Entren.	Valid.	Brecha	Lectura
Logística L2 (lambda=100)	0.4137	0.4129	0.0008	Sesgo alto: casi sin brecha, pero 0.026 por debajo del boosting; una suma lineal no capta relaciones no lineales ni interacciones
Logística L1 (lambda=10)	0.4133	0.4124	0.0009	Sesgo alto: casi sin brecha, pero 0.026 por debajo del boosting; una suma lineal no capta relaciones no lineales ni interacciones
Logística Elastic Net 0.5 (lambda=31.62)	0.4134	0.4126	0.0008	Sesgo alto: casi sin brecha, pero 0.026 por debajo del boosting; una suma lineal no capta relaciones no lineales ni interacciones
HGB lr=0.1, hojas=15	0.4884	0.4388	0.0495	Equilibrado: brecha pequeña y la mejor validación; árboles limitados y parada temprana controlan la varianza
Random Forest (mínimo 50 por hoja)	0.4669	0.4226	0.0442	Equilibrado: brecha pequeña y la mejor validación; árboles limitados y parada temprana controlan la varianza
Random Forest (hojas de 1, sin podar)	0.9970	0.3705	0.6265	Varianza alta: memoriza el entrenamiento (0.997) y en validación queda por debajo de la logística

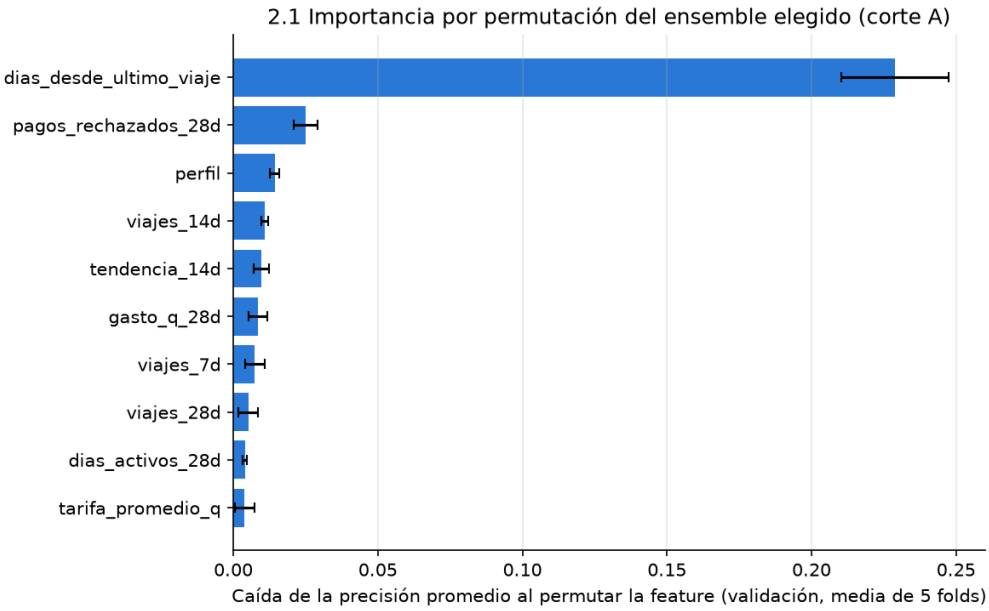


Importancias por permutación

Se calcularon en el fold de validación de cada uno de los 5 modelos del boosting elegido (5 repeticiones por feature) y se promediaron. La importancia es cuánto cae la precisión promedio al desordenar la feature en datos que el modelo no vio.

#	Feature	Caída de la precisión promedio	Desv. entre folds
1	dias_desde_ultimo_viaje	0.2290	0.0186
2	pagos_rechazados_28d	0.0251	0.0042
3	perfil	0.0143	0.0015
4	viajes_14d	0.0109	0.0011
5	tendencia_14d	0.0097	0.0028
6	gasto_q_28d	0.0085	0.0031
7	viajes_7d	0.0074	0.0034
8	viajes_28d	0.0052	0.0033
9	dias_activos_28d	0.0040	0.0008
10	tarifa_promedio_q	0.0039	0.0033

Días desde el último viaje domina (0.229), igual que en los coeficientes de la logística y que en la línea base 2: quien ya lleva días sin viajar es quien no vuelve. Le siguen los pagos rechazados de Transurbano (un problema de pago que empuja a dejar la tarjeta), el perfil del padrón y la caída reciente de viajes (viajes de 14 días y tendencia).



Decisión

Va a producción el HistGradientBoosting (learning_rate 0.1, 15 hojas). Mejora la precisión promedio de la logística en 0.0259 (de 0.4129 a 0.4388), unas 2.3 desviaciones estándar entre folds, y gana en los 5 folds. En dinero, con el mismo escenario de 2.2, deja Q12,155 en el corte A contra Q10,875 de la logística (+12 %). La complejidad extra es poca en la práctica: entrena en un segundo, se guarda igual con joblib dentro del mismo Pipeline y la importancia por permutación explica qué usa. Lo que perdemos es leer coeficientes directamente; por eso la logística queda documentada como respaldo interpretable.

2.2 Umbral según costo y test único

Ganancia

Escenario de la Agencia: contactar cuesta Q5; si el usuario se iba a ir, la campaña lo retiene en el 25 % de los casos; un usuario retenido vale Q60.

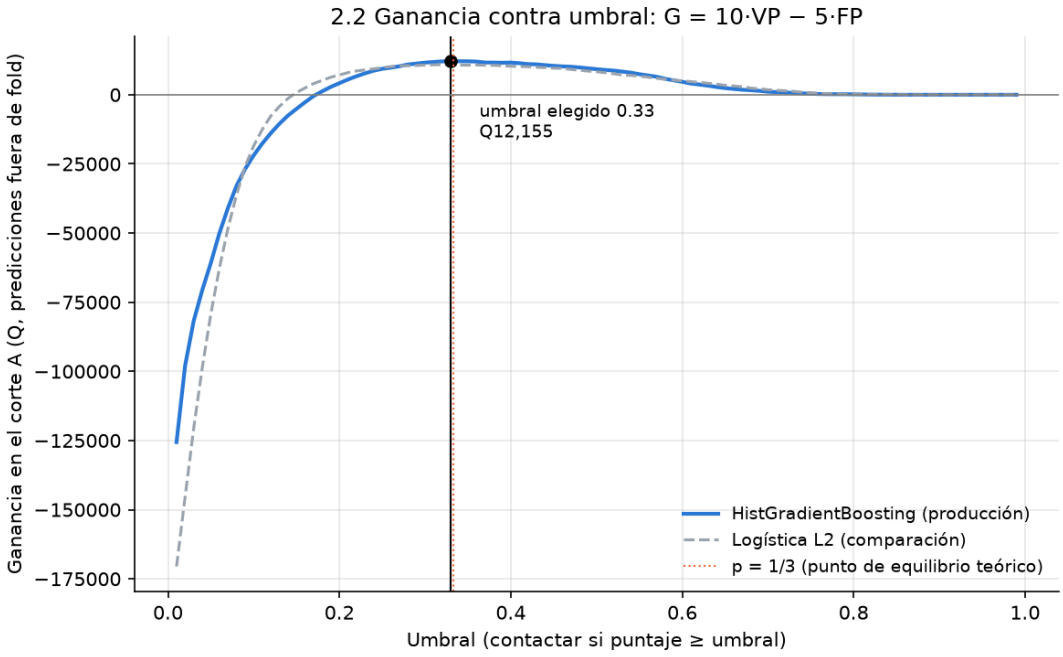
Contactar a alguien que sí se iba deja en promedio 0.25 × 60 − 5 = 15 − 5 = **Q10** (verdadero positivo). Contactar a alguien que no se iba cuesta **Q5** (falso positivo). A quien no contactamos no le ganamos ni le perdemos nada. Para un grupo contactado:

`G = 10 · VP − 5 · FP`

Para una persona con probabilidad p de irse, contactarla conviene si 0.25 · 60 · p − 5 > 0, es decir si p > 5 / 15 = 1/3. Ese es el umbral teórico si las probabilidades del modelo están bien calibradas.

Curva y umbral elegido

Calculamos G para umbrales de 0.01 a 0.99 con las predicciones fuera de fold del corte A (cada persona predicha por el modelo que no la vio). El máximo está en **0.33**, justo el 1/3 teórico: el boosting está bien calibrado. Con ese umbral se contacta a 6,014 personas de 59,491: G = 10 · 2,815 − 5 · 3,199 = 28,150 − 15,995 = Q12,155. Precisión 0.4681, recall 0.3968. Contactar a todos daría 10 · 7,095 − 5 · 52,396 = Q-191,030. Por fold, el óptimo va de 0.33 a 0.38.



Test B

Después de elegir el modelo y el umbral solo con A, entrenamos el modelo final con todo el corte A (modelos/modelo_abandono_v1.0.0.joblib, 103 árboles) y lo evaluamos **una sola vez** en el corte B con proyecto2/test_b.py, que exige un commit limpio y se niega a correr si ya existe el registro.

Qué medimos	Resultado
Fecha y commit de la evaluación	2026-11-08T12:21:45-06:00 · commit 556830f
Población B	6,387 / 58,113 = 0.1099
Precisión promedio	0.3905 (línea base días desde el último viaje: 0.3482; constante: 0.1099)
ROC-AUC	0.8429 (línea base: 0.8173)
Umbral	0.33
Precisión / recall	0.4208 / 0.4154
Ganancia	G = 10 · 2,653 − 5 · 3,652 = 26,530 − 18,260 = Q8,270
Contactar a todos	10 · 6,387 − 5 · 51,726 = Q-194,760

Matriz de confusión en B	Contactar (puntaje ≥ 0.33)	No contactar
Real: se fue (1)	VP = 2,653	FN = 3,734
Real: siguió (0)	FP = 3,652	VN = 48,074

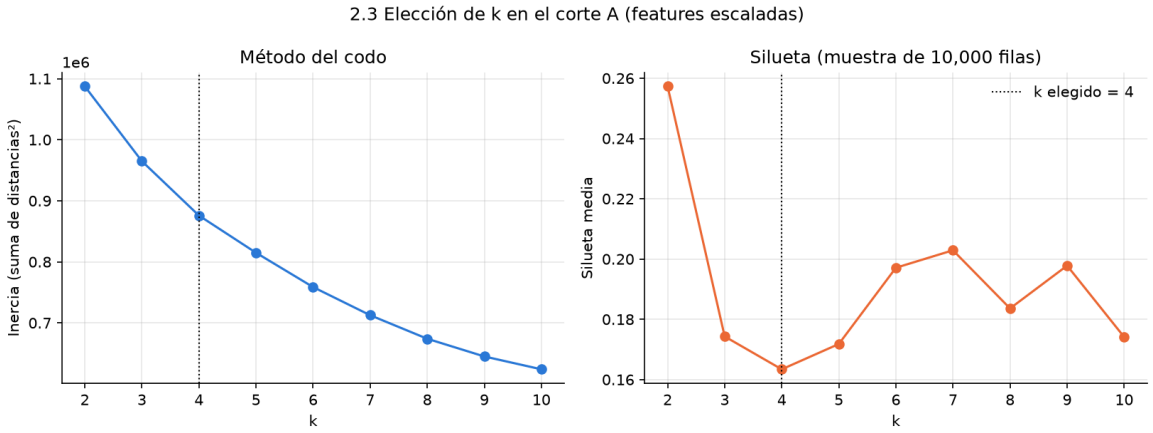
El resultado es peor que en validación (precisión promedio 0.3905 contra 0.4388; ganancia de Q142 contra Q204 por cada 1,000 personas) y no volvimos a ajustar nada. La explicación está en el mes, no en un sobreajuste: en B la tasa de abandono bajó de 0.1193 a 0.1099, y la precisión promedio depende de cuántos positivos hay; la línea base, que no aprende nada, cayó en la misma proporción (de 0.3892 a 0.3482, −10.5 %; el modelo −11 %). El ROC-AUC, que no depende de la tasa, casi no cambió (0.8446 → 0.8429): el modelo ordena igual de bien. Con menos positivos, el mismo umbral deja más falsos positivos (precisión 0.421 contra 0.468), pero la campaña sigue siendo rentable y el modelo supera a la línea base.

2.3 Segmentación con KMeans y PCA

Aplicamos KMeans a las 22 features numéricas del corte A, escaladas con StandardScaler (sin escalar, el gasto y los viajes, que van en decenas, dominarían a las proporciones de 0 a 1). El perfil del padrón es categórico y queda fuera; su composición por segmento está en 2.3-perfil-por-segmento.csv. El segmento no se usa como feature del clasificador, así que no hace falta ajustar KMeans dentro de la validación cruzada.

Elección de k

k	Inercia	Caída de la inercia	Silueta
2	1,087,449		0.2573
3	964,908	11.3 %	0.1743
4	875,808	9.2 %	0.1634
5	814,893	7.0 %	0.1718
6	758,657	6.9 %	0.1971
7	712,564	6.1 %	0.2029
8	674,022	5.4 %	0.1836
9	645,013	4.3 %	0.1978
10	623,968	3.3 %	0.1741



Elegimos k = 4, aunque la silueta máxima es k = 2, y explicamos por qué. Con k = 2 solo se separan los usuarios intensivos del resto (abandono 2 % contra 14 %), que no alcanza para decidir acciones. Los picos de silueta en k = 6 y 7 separan por operador (Transmetro, Transurbano, MetroRiel, Aerómetro) y esos grupos tienen casi la misma tasa de abandono (13 % a 15 %), así que no sirven para retener. El codo está en k = 4: hasta ahí cada grupo más baja la inercia 11 % y 9 %, y después 7 % o menos. Con k = 4 los segmentos tienen tasas de abandono muy distintas (21 %, 14 %, 4 % y 2 %). La silueta de todos los k es baja (0.16 a 0.26): los usuarios forman un continuo y no grupos muy separados.

Gráfico PCA

Proyectamos los datos escalados a dos componentes principales: la primera explica 29.5% y la segunda 12.1% (en total 41.5%). La primera crece con la intensidad de uso (viajes, días activos, gasto); la segunda separa a los de Aerómetro.

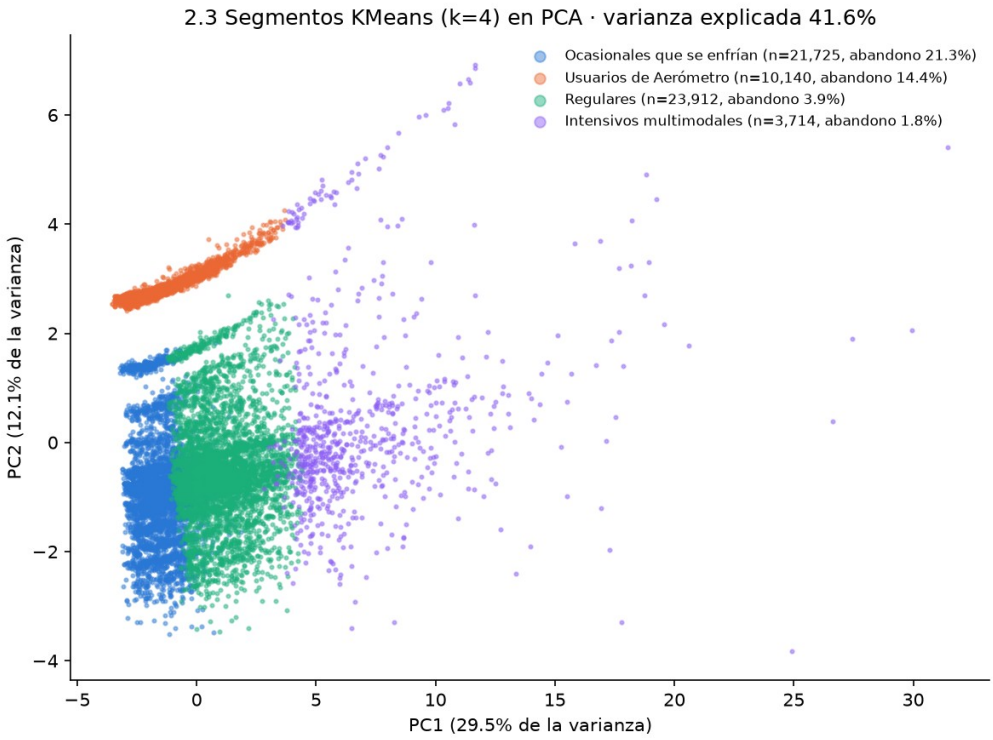


Tabla de segmentos

Segmento	Tamaño	Viajes 28 d	Días desde último	Días activos	Prop. Aerómetro	Pagos rechaz.	Gasto Q	Tasa de abandono
Ocasionales que se enfrían	21,725 (36.5%)	5	6	4	0	0	6	4,636 / 21,725 = 0.2134
Usuarios de Aerómetro	10,140 (17.0%)	6	4	5	1	0	21	1,459 / 10,140 = 0.1439
Regulares	23,912 (40.2%)	17	2	11	0	0	22.1	933 / 23,912 = 0.0390
Intensivos multimodales	3,714 (6.2%)	63	1	22	0	1	92.3	67 / 3,714 = 0.0180

Medianas de cada segmento. La tasa de abandono es descriptiva: dice dónde se concentran los que se van, no que el segmento cause el abandono.

Acciones

Segmento	Acción de retención
Ocasionales que se enfrían	Recarga de cortesía (la campaña) solo a quienes el modelo pone sobre el umbral: aquí se concentran los abandonos y cada contacto acertado deja Q10.
Usuarios de Aerómetro	No regalar recarga: viajan por vuelos y su "abandono" suele ser falta de viaje, no pérdida. Mensaje de bajo costo antes del regreso y mejorar el enlace de su tarjeta con la persona.
Regulares	No contactar con recarga (abandono bajo). Encuesta corta o aviso de cambios de ruta para mantenerlos.
Intensivos multimodales	Atender los pagos rechazados (recarga automática o alerta de saldo bajo) en lugar de descuentos: casi no abandonan, pero son los que más gastan.

2.4 MLOps: predecir, guardar y documentar

predecir.py

```
python predecir.py --datos datos_red --fecha-corte 2026-08-30
```

Carga modelos/modelo_abandono_v1.0.0.joblib, que contiene el Pipeline completo (preprocesamiento y modelo), el umbral, la lista de features y la versión; no llama a fit. Para las features necesita Silver: si el warehouse del proyecto se construyó con exactamente esos archivos (compara el MD5 de cada uno con control.ingest_log) lo usa, y si no corre el flujo del Proyecto 1 en .prediccion/ sin tocar el warehouse principal. Después calcula la población y las 23 features con el mismo código de la Fase 1 (proyecto2/features.py) y escribe el CSV con unidad_id, tarjetas (llaves originales vistas en los 28 días antes del corte, separadas por ;), puntaje y contactar (1 si el puntaje es al menos 0.33).

Predicciones del corte C

predicciones/predicciones_corte_2026-08-30.csv: 57,701 filas, una por persona de la población de C; 7,268 con contactar = 1. Según los puntajes del modelo, la ganancia esperada de contactarlos es Q17,177 (suma de 15·p − 5 de cada contactado). SHA-256 del CSV: 952923b12e0609c6…

Modelo guardado y entorno

modelos/modelo_abandono_v1.0.0.joblib (la versión va en el nombre) y su .json con la configuración, la fecha y el commit de entrenamiento y su SHA-256. Entorno: Python 3.12, requirements.txt con versiones fijas (scikit-learn 1.9.1, pandas 3.1.0, duckdb 1.5.5, prefect 3.8.6 y sus dependencias), semilla 2026 en folds, modelos y KMeans. Fijamos fastapi 0.150.0 porque la 0.156 o más nueva rompe el servidor temporal de prefect 3.8.6. El README explica cómo correrlo en otra máquina.

Los scripts de esta fase leen las features de entregables/proyecto2/features-cortes.csv, que está en git, así que cualquier commit repite sus cifras en otra máquina. unidad_id es un seudónimo que depende de PSEUDO_SALT (.env): las predicciones se generan y se comparan con el mismo .env.

Ficha del modelo

docs/proyecto2/ficha-modelo.md, en una página: datos y periodo, unidad, población, etiqueta, métricas en validación y en B, umbral y su costo, fugas revisadas, limitaciones y el aviso de que los datos son simulados.

Ensayo de la prueba de invariancia

El día de la defensa recibiremos 30 días más de datos. Ensayamos lo mismo con el corte B, que ya tiene 30 días después: corrimos predecir.py con fecha 31 de julio sobre los datos completos (hasta el 29 de agosto) y sobre los 9 archivos recortados en binario a lo anterior al 31 de julio. Los dos CSV son idénticos byte a byte (SHA-256 09c662cb5d3eedb7… en ambos). También repetimos la predicción de C y salió idéntica a la entregada. Las tarjetas salen solo de la ventana anterior al corte por esta razón: si se tomaran de todos los datos, una tarjeta nueva en septiembre cambiaría el CSV.

Métricas que reportamos

Categoría	Resultado
Problema	A: 7,095 / 59,491 = 0.1193 · B: 6,387 / 58,113 = 0.1099 · C: 57,701 sin etiqueta
Validación	Precisión promedio, porque los positivos son ~11 % y la campaña contacta primero a los de más riesgo. Boosting 0.4388 ± 0.0113; línea base 0.3892 ± 0.0078; constante 0.1193
Decisión	Umbral 0.33; en A precisión 0.4681, recall 0.3968, Q12,155; en B precisión 0.4208, recall 0.4154, Q8,270
Test	Corte B, 2026-11-08 12:21, commit 556830f: precisión promedio 0.3905 (línea base 0.3482)
Fuga	Prueba de recorte: 0 diferencias en 59,491 filas (Fase 1). Ensayo de invariancia: CSV idéntico con 30 días más
Rendimiento	Carga desde cero hasta Silver/Gold 280 s; features de los 3 cortes 9.0 s; entrenamiento del modelo final 1.01 s (validación cruzada del boosting: 3.4 s); predicción de C 0.151 s (predecir.py completo con Silver lista: 4.65 s)

Todos los tiempos se midieron en una laptop; la carga desde cero es la de la corrida final de la Fase 1 (entregables/proyecto2/1.1-tiempos.csv). Cada corrida está en proyecto2/bitacora.md con fecha y commit.

Los datos son simulados.